

Standardized Benchmarking of Multi-Agent Distributed Machine Learning in Augmented Reality

Research Update #4

Alexis Juarez Gomez & Parthkumar Joshi

1. Wiring Mockups

This week, one of the tasks we did was take our static UI mockups in the prototype (the dashboard, deck browser, AR scan view, and flashcard detail page) and wired them together so every clickable element now opens the correct page with real content. Previously only three flashcards (copper sulfate, mitochondria, and Hagia Sophia) had populated detail data, which meant that any other deck card, recent-scan tile, or due-today item silently fell through to a copper-sulfate fallback. I authored ten additional flashcards spanning chemistry, biology, botany, architecture, geology, art history, physics, and astronomy, each with a subject eyebrow, formula, spaced-repetition metadata, a one-line summary, and four numbered facts that match the existing editorial voice. I also refactored the duplicated deck-to-card mapping logic that was scattered across the hash router into a single deck to card lookup, so adding a new deck now only requires editing one place. The result is that every clickable element (deck cards, recent scans, due-today items, and the 🔍 search palette) routes to its correct representative flashcard. I verified the flow by running the prototype on a local server and clicking through every entry point, then committed the working version to the GitHub repo.

2. Testing & Benchmarking the YOLOv8 model

This week was more focused on training the model and familiarizing ourselves with it. I uploaded 591 stock images containing objects that are commonly found in schools, such as textbooks, desks, chairs, chemical flasks, computer labs, etc. We are using Roboflow to annotate our own dataset. Since the process is long and tedious, we are still working on the dataset, but have made substantial progress on bounding box annotation and class labeling across the dataset. I made and ran a benchmarking script to evaluate the pretrained YOLOv8n model as a performance baseline on Apple M2 hardware. Running on the MPS (Metal Performance Shaders) backend with a 640px input size and 0.25 confidence threshold, the model processed all 591 images and recorded an average inference latency of 215.3 ms/image, yielding 4.6 FPS with an average detection confidence of 51.07%. This is currently below our real-time target of ≥ 15 FPS, which is expected given that this is a general-purpose COCO-pretrained model that has not yet been fine-tuned on our school object dataset. We aim for ≥ 15 FPS, which represents the minimum acceptable responsiveness without requiring dedicated GPU acceleration for an AR environment. These baseline numbers will serve as a direct comparison point once custom training is complete in the next Research Progress update.

The images attached below are the results of the benchmark. Since the model has not been fully trained to where we need, the code will still be separate from the frontend; however, the benchmark script and the result CV will be committed to the GitHub repo as a reference to when the models are fully trained.

```
Device   : MPS
Model    : ultralytics/yolov8n.pt
Img size : 640px
Conf     : 0.25
```

```
Images   : 591 across 6 classes
```

```
Warming up model... WARNING ⚠ NMS time limit 2.050s exceeded
done
```

Class	Images	Avg ms	FPS	Avg dets	Avg conf
textbook	99	584.1ms	1.7	5.70	43.56%
whiteboard	98	201.6ms	5.0	4.15	55.17%
desk_chair	99	112.5ms	8.9	4.86	48.88%
chemistry_flask	98	122.4ms	8.2	3.01	56.36%
laptop_computer	99	140.5ms	7.1	3.41	58.87%
ruler_pencil_pen	98	128.8ms	7.8	1.37	46.78%
OVERALL	591	215.3ms	4.6		51.07%

BENCHMARK SUMMARY

```
Model       : YOLOv8n (pretrained COCO baseline)
Hardware    : Apple M2 (MPS backend)
Dataset     : 591 school object images (6 classes)
Avg latency : 215.3 ms/image
Avg FPS     : 4.6 frames/sec
Avg conf    : 51.07%
```